

Vulkan Notes

Vertex Buffer

Adam HAMOUC

June 2026

1. Vertex Input Description

Les données des vertices sont définies côté CPU dans un tableau de structs. Pour que Vulkan sache comment lire ce tableau, on fournit deux descriptions complémentaires.

Vertex Binding Description — décrit le buffer entier. Elle indique le stride (combien d'octets entre deux vertices consécutifs) et le rythme d'avance. Elle ne dit pas ce que contiennent les données, juste comment les sectionner.

```
VkVertexInputBindingDescription binding{};
binding.binding = 0;
binding.stride = sizeof(Vertex); // octets entre deux vertices
binding.inputRate = VK_VERTEX_INPUT_RATE_VERTEX;
```

Vertex Attribute Description — décrit chaque champ à l'intérieur d'un vertex. Elle fait le lien entre un champ du struct CPU et une variable `layout(location=x)` dans le vertex shader, en précisant son format et son offset dans le bloc.

```
// Position : vec2, au debut du struct
attributeDescriptions[0].binding = 0;
attributeDescriptions[0].location = 0; // location=0 dans le shader
attributeDescriptions[0].format = VK_FORMAT_R32G32_SFLOAT;
attributeDescriptions[0].offset = offsetof(Vertex, pos); // 0

// Couleur : vec3, juste apres pos (offset = 8 octets)
attributeDescriptions[1].binding = 0;
attributeDescriptions[1].location = 1; // location=1 dans le shader
attributeDescriptions[1].format = VK_FORMAT_R32G32B32_SFLOAT;
attributeDescriptions[1].offset = offsetof(Vertex, color); // 8
```

Formats courants (les noms correspondent à des composantes de couleur par convention) :

Type GLSL	Format Vulkan	Taille
float	VK_FORMAT_R32_SFLOAT	4 o
vec2	VK_FORMAT_R32G32_SFLOAT	8 o
vec3	VK_FORMAT_R32G32B32_SFLOAT	12 o
vec4	VK_FORMAT_R32G32B32A32_SFLOAT	16 o
ivec2	VK_FORMAT_R32G32_SINT	8 o
uvec4	VK_FORMAT_R32G32B32A32_UINT	16 o
double	VK_FORMAT_R64_SFLOAT	8 o

Le type (SFLOAT, SINT, UINT) et le nombre de bits doivent correspondre exactement au shader. Le nombre de composantes peut être supérieur — les extras sont silencieusement ignorés. Il ne peut pas être inférieur pour la première composante.

2. Vertex Buffer

Un vertex buffer est un amas de memoire GPU decrivant les vertices. En Vulkan, creation du buffer et allocation memoire sont deux etapes separees.

Etape 1 — Creer le buffer (`vkCreateBuffer`) : cree un handle qui decrit ce qu'on veut (taille, usage). Aucune memoire n'est encore reservee.

Etape 2 — Verifier les requirements (`vkGetBufferMemoryRequirements`) : interroge le driver pour connaitre la taille reelle necessaire (peut etre superieure a cause de l'alignement), l'alignement requis, et un bitmask des types de memoire compatibles.

Etape 3 — Choisir le type de memoire (`findMemoryType`) : parcourt les types de memoire disponibles sur le physical device pour trouver un type compatible avec le buffer ET qui possede les proprietes demandees (`HOST_VISIBLE`, `DEVICE_LOCAL`, etc.).

Etape 4 — Allouer (`vkAllocateMemory`) : reserve la memoire sur le GPU.

Etape 5 — Binder (`vkBindBufferMemory`) : attache la memoire allouee au buffer.

Types de memoire courants :

Flags	Usage
<code>HOST_VISIBLE HOST_COHERENT</code>	CPU peut ecrire, staging buffer, UBOs
<code>DEVICE_LOCAL</code>	VRAM pure, lecture GPU rapide
<code>HOST_VISIBLE HOST_CACHED</code>	Readback GPU vers CPU

`HOST_COHERENT` : ecritures CPU visibles immediatement sans flush manuel.

`HOST_CACHED` : lectures CPU plus rapides (cache), mais flush/invalidate manuel requis.

Limite d'allocations : `vkAllocateMemory` est limitee (ex. 4096 appels sur GTX 1080). En production on alloue une grande zone et on gere des offsets en interne — c'est ce que fait **VulkanMemoryAllocator (VMA)**. A integrer dans la RHI derriere `IRHIBuffer`.

3. Staging Buffer

Allouer le vertex buffer directement en `HOST_VISIBLE` fonctionne mais n'est pas optimal. La memoire partagee CPU/GPU est plus lente pour le GPU que la VRAM pure (`DEVICE_LOCAL`). Sur un GPU discret ces deux types sont physiquement incompatibles — on ne peut pas avoir les deux flags sur le meme buffer.

Solution : le staging buffer.

1. Creer un buffer temporaire en `HOST_VISIBLE | HOST_COHERENT` (accessible CPU)
2. Mapper, `memcpy` les vertices dedans, unmapper
3. Creer le vertex buffer final en `DEVICE_LOCAL` (VRAM pure)
4. `vkCmdCopyBuffer` : le GPU copie du staging vers le buffer final
5. Detruire le staging buffer

```
// Staging : CPU peut ecrire
createBuffer(size, VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
             VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
             stagingBuffer, stagingMemory);

void* data;
vkMapMemory(device, stagingMemory, 0, size, 0, &data);
memcpy(data, vertices.data(), size);
vkUnmapMemory(device, stagingMemory);

// Vertex buffer final : GPU only
createBuffer(size,
             VK_BUFFER_USAGE_TRANSFER_DST_BIT | VK_BUFFER_USAGE_VERTEX_BUFFER_BIT,
             VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT,
             vertexBuffer, vertexMemory);

copyBuffer(stagingBuffer, vertexBuffer, size); // vkCmdCopyBuffer
```

Quand utiliser un staging buffer ?

Staging + <code>DEVICE_LOCAL</code>	<code>HOST_VISIBLE</code> direct
Vertex / index buffers (upload unique)	UBOs (mis a jour chaque frame)
Textures	Push constants
Meshes statiques	Donnees qui changent souvent

Regle : upload unique → staging. Mise a jour frequente → `HOST_VISIBLE` direct.

Compatibilite `memcpy` : le struct `Vertex` doit etre standard layout — pas de vtable, pas de padding surprenant. Verifier avec `static_assert(offsetof(Vertex, color) == 8, "padding inattendu")`. Les types `glm::vec2/vec3` sont garantis packed et alignes correctement pour le GPU.